

Clockless Federated Adaptation

Asynchronous Medical-Image Federated Learning Without a Global Clock

Group 4: Asynchronous Distributed ML Adaptation

Distributed Systems Final Proposal Deck

02 01 Motivation | Start from Hospital Collaboration

Multiple hospitals want to train a shared medical-image model, but raw patient images cannot be centralized.

- Hospitals have different hardware, networks, and workloads.
- Some updates arrive quickly; others arrive much later.
- Patient populations and image distributions differ across hospitals.

Question: async training is faster because it does not wait for slow hospitals, but can late updates still be trusted?

03 01 Motivation | The Core Distributed-Systems Tension

Sync FL: wait for all clients -> the slowest node determines progress
Async FL: do not wait -> receive stale / out-of-order updates

This project focuses on:

throughput vs consistency / convergence correctness

This is a distributed-systems problem about time, ordering, and consistency, not only an image-classification problem.

04 02 Preliminary | Sync vs Async FL

Setting	System behavior	Main risk
Sync FedAvg	round barrier; wait for clients	straggler bottleneck
Naive Async	apply each update on arrival	stale-update instability
Buffered Async	aggregate after B updates	conflicts may still exist inside the buffer

05 02 Preliminary | Logical Staleness

We do not assume a synchronized physical clock.

```
server_version = current global model version  
client_start_version = version received by the client  
staleness = server_version - client_start_version
```

This is the FL-simulator counterpart of Lamport logical time: logical model versions replace wall-clock ordering.

06 03 Problem Formulation | Fair Comparison

Async may look worse simply because it used fewer client updates.

```
Sync update budget  = rounds x clients  
Async update budget = events  
Fair comparison     = events = rounds x clients
```

Official experiment matrix:

```
9 datasets x 6 methods x 3 seeds = 162 official runs
```

07 03 Problem Formulation | Evaluation Metrics

ML metrics:

```
best_acc, final_acc, stability_drop = best_acc - final_acc
```

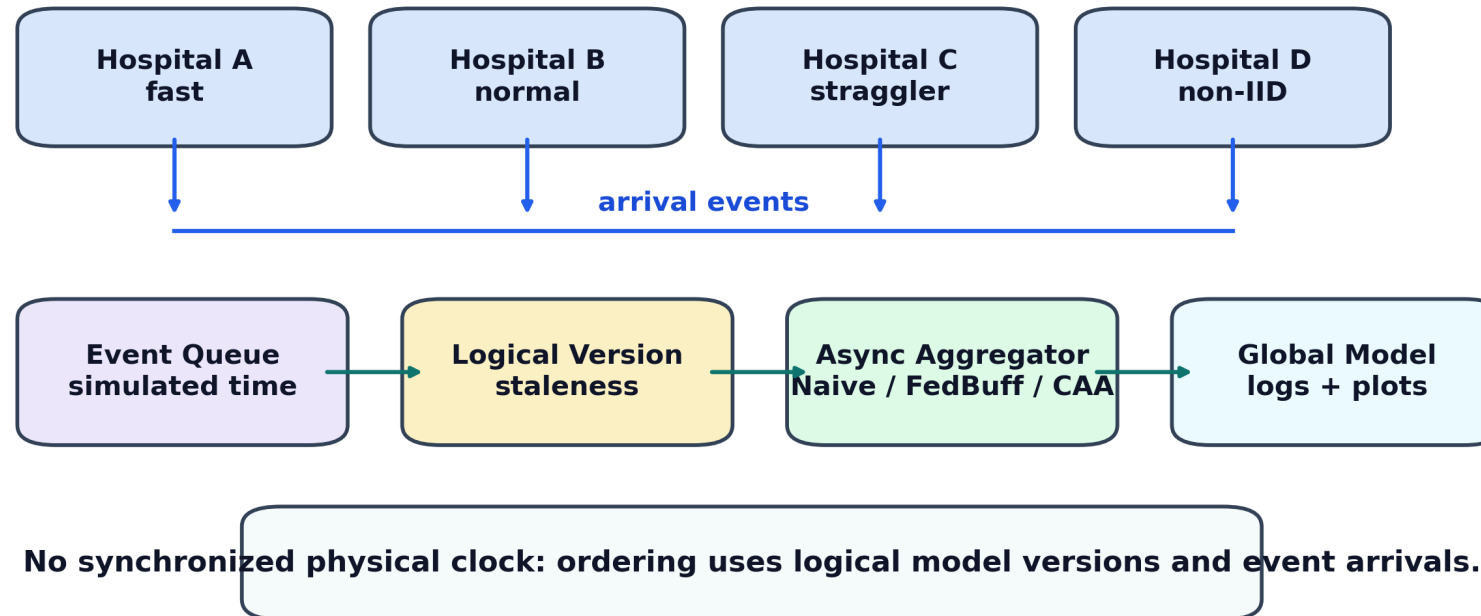
Distributed-systems metrics:

```
p95 staleness, simulated time, effective alpha,  
client contribution Gini, time-to-accuracy
```

Accuracy tells us what happened; staleness / delay / imbalance explain why.

08 03 Architecture | Event-Driven Clockless Simulator

Event-Driven Clockless FL Simulator

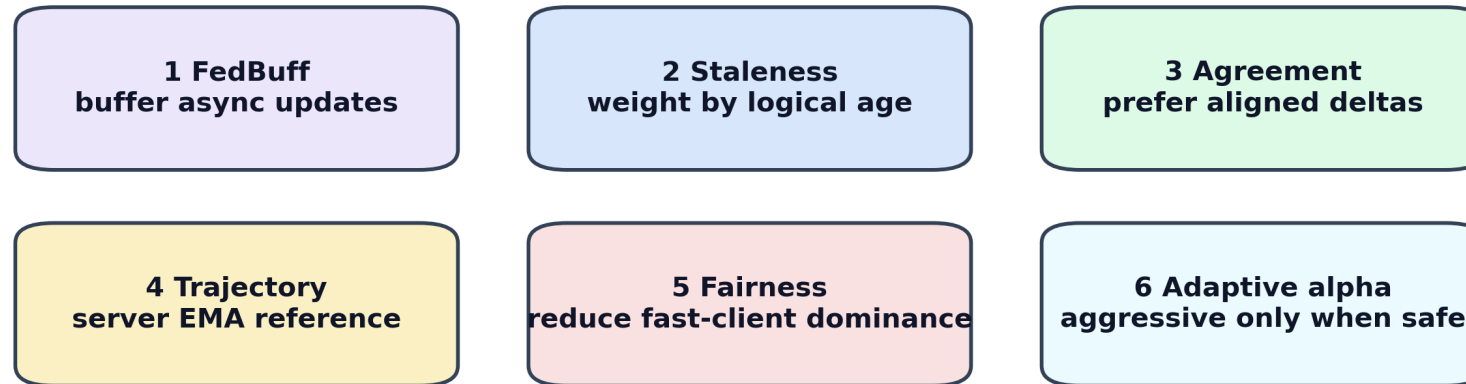


09 04 Methodology | Baseline Map

Method	Role	What it tests
Sync	upper reference	no async staleness, but has a barrier
Naive Async	stateless async	throughput without correction
Staleness	logical-age correction	whether age alone is enough
FedBuff	buffered async baseline	whether buffering stabilizes
CAA-v1	agreement-aware	whether direction helps
CAA-v2	final method	direction + trajectory + fairness

10 04 Methodology | CAA-v2 Is More Than FedBuff

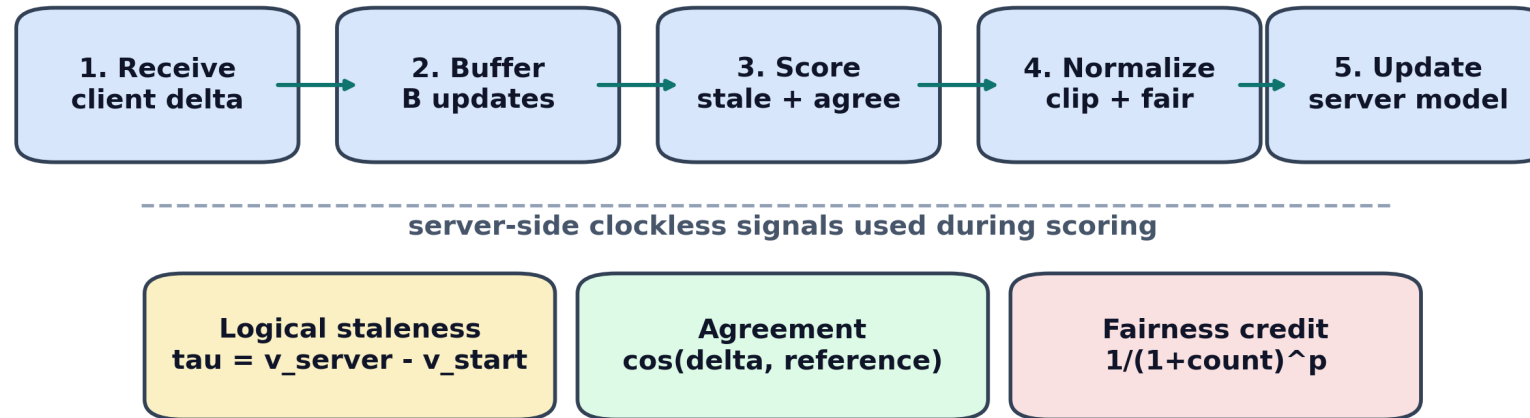
What CAA-v2 Adds Over FedBuff



Simple server rule: combine known signals into one clockless async aggregation policy.

11 04 Methodology | CAA-v2 Server Pipeline

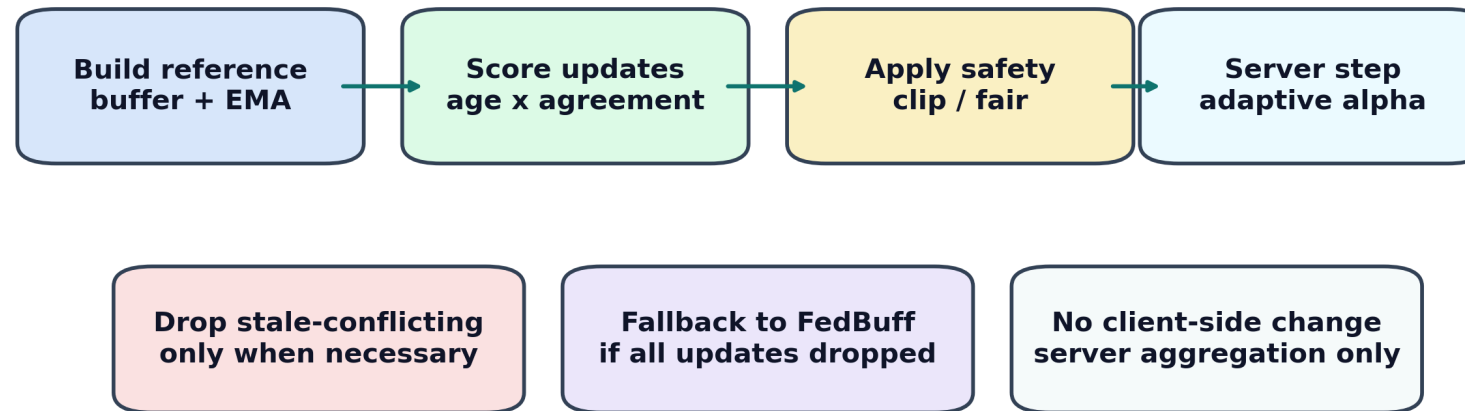
CAA-v2 Aggregation Pipeline



All decisions use logical versions, deltas, accepted trajectory, and contribution counts.

12 04 Methodology | Server Decision Flow

CAA-v2 Server Decision Flow



The flow is deterministic and clockless; no physical timestamp is required.

13 04 Methodology | Core Equations

For each client update:

```
delta_i = client_model_i - model_at_client_start_i  
tau_i   = server_version - client_start_version_i
```

CAA-v2 weight:

```
raw_weight_i = n_i x staleness_decay(tau_i)  
              x agreement(delta_i, ref) x fairness_i
```

Server update:

```
w <- w + alpha_buffer x weighted_average(delta_i)
```

14 04 Methodology | Reference Direction

```
buffer_ref = weighted_average(delta_i, n_i x age_i)
server_ema = EMA(previous accepted server deltas)
ref        = blend(buffer_ref, server_ema)
```

Intuition:

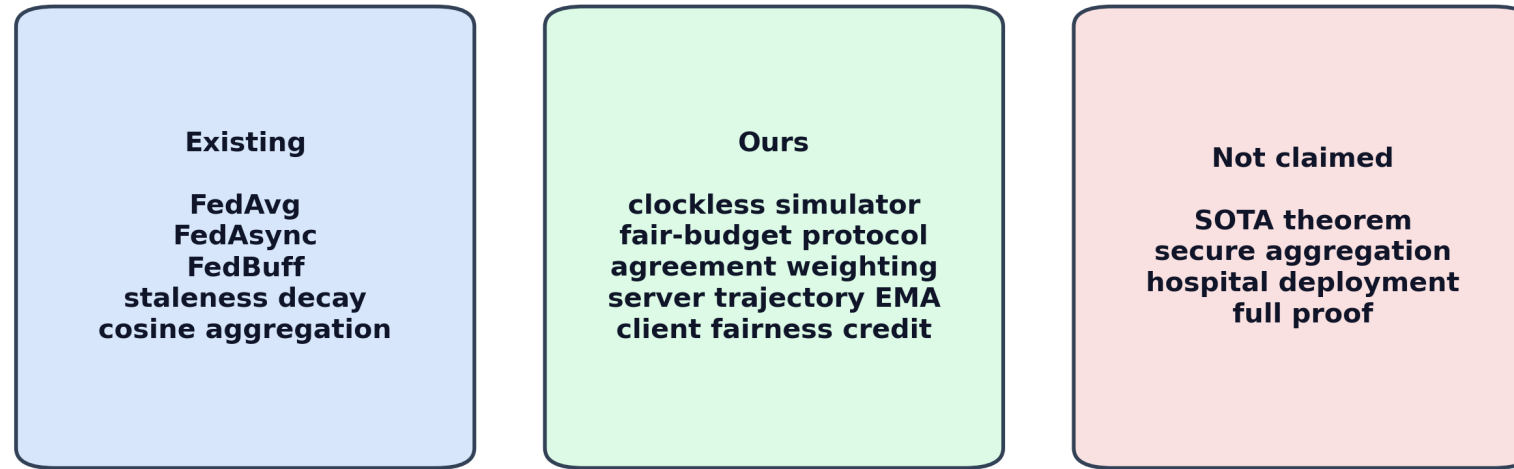
- `buffer_ref`: the group direction of the current buffered updates.
- `server_ema`: where the global model has recently been moving.
- A stale update pointing against this direction should not be blindly amplified.

15 04 Methodology | Clockless Signals

Signal	Source	Uses physical clock?
staleness	server/client logical version	No
agreement	delta dot product	No
server trajectory	accepted server deltas	No
fairness credit	accepted contribution count	No
adaptive alpha	mean staleness + agreement	No

16 04 Methodology | Novelty Boundary

Novelty Boundary



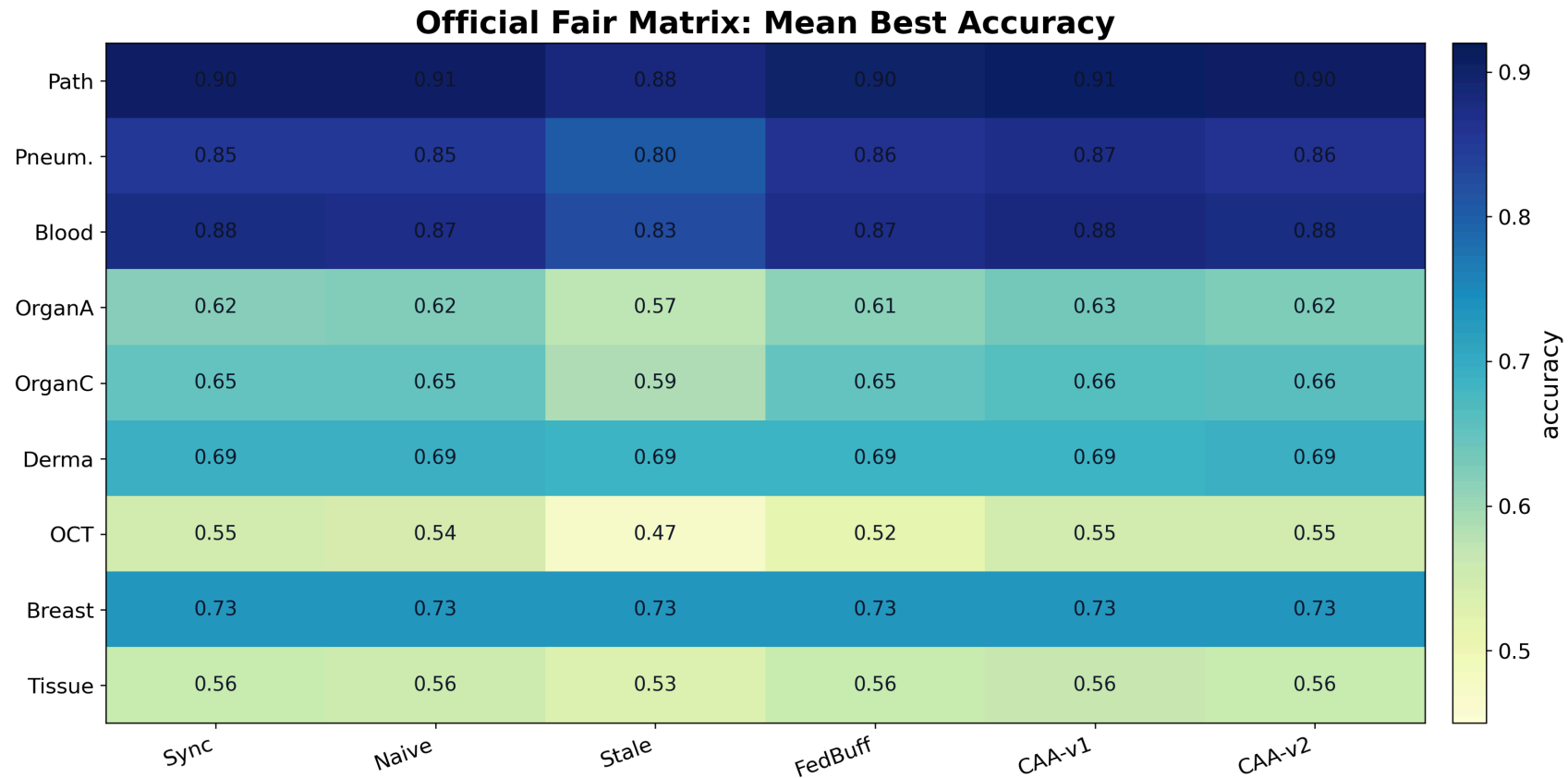
Contribution: reproducible systems integration and evidence, not a wholly new FL primitive.

17 05 Experiment Results | Fairness Protocol

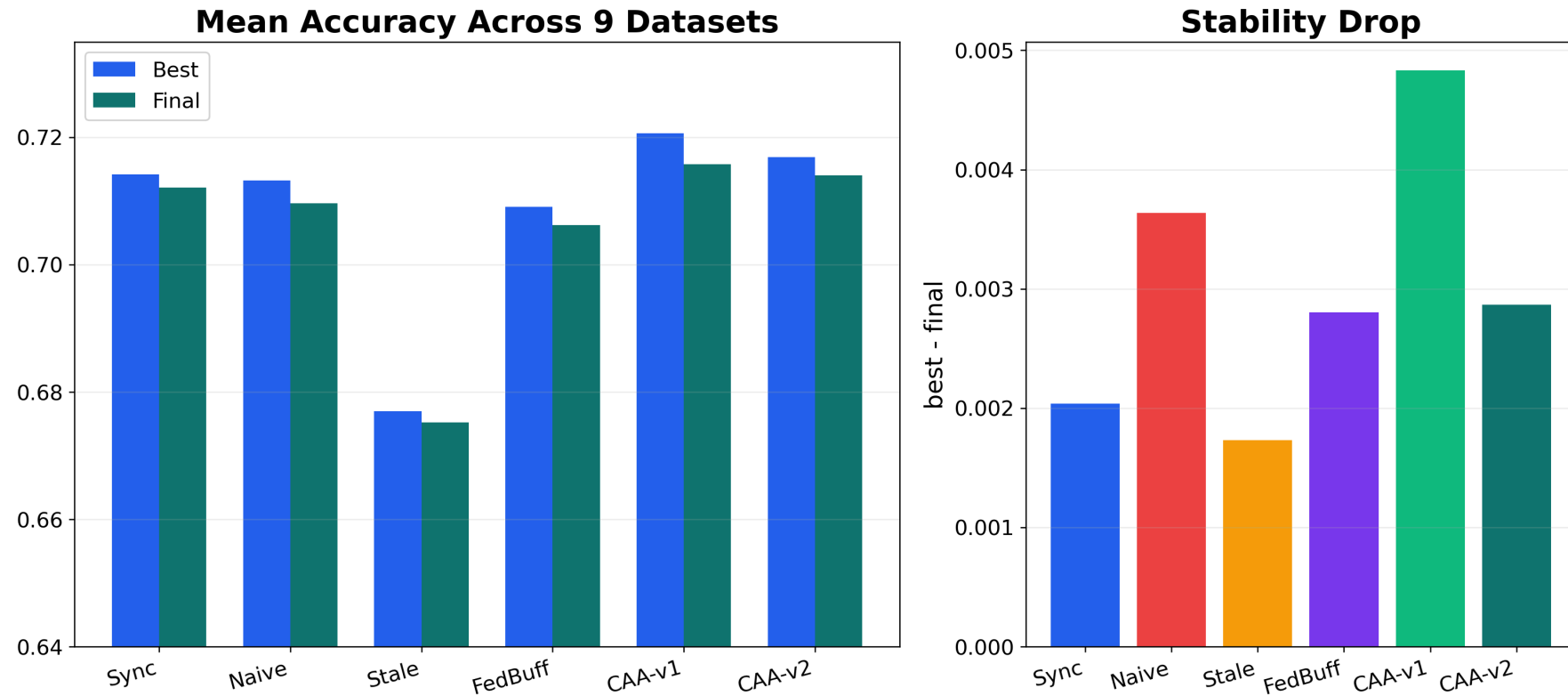
```
clients = 10
local_epochs = 1
batch_size = 128
lr = 0.01, cosine scheduler
seeds = 42, 43, 44
async delay = same heterogeneous setting
fair budget = async events = sync rounds x clients
```

Official comparisons do not mix different backbones, delay settings, seeds, local epochs, or update budgets.

18 05 Experiment Results | Coverage

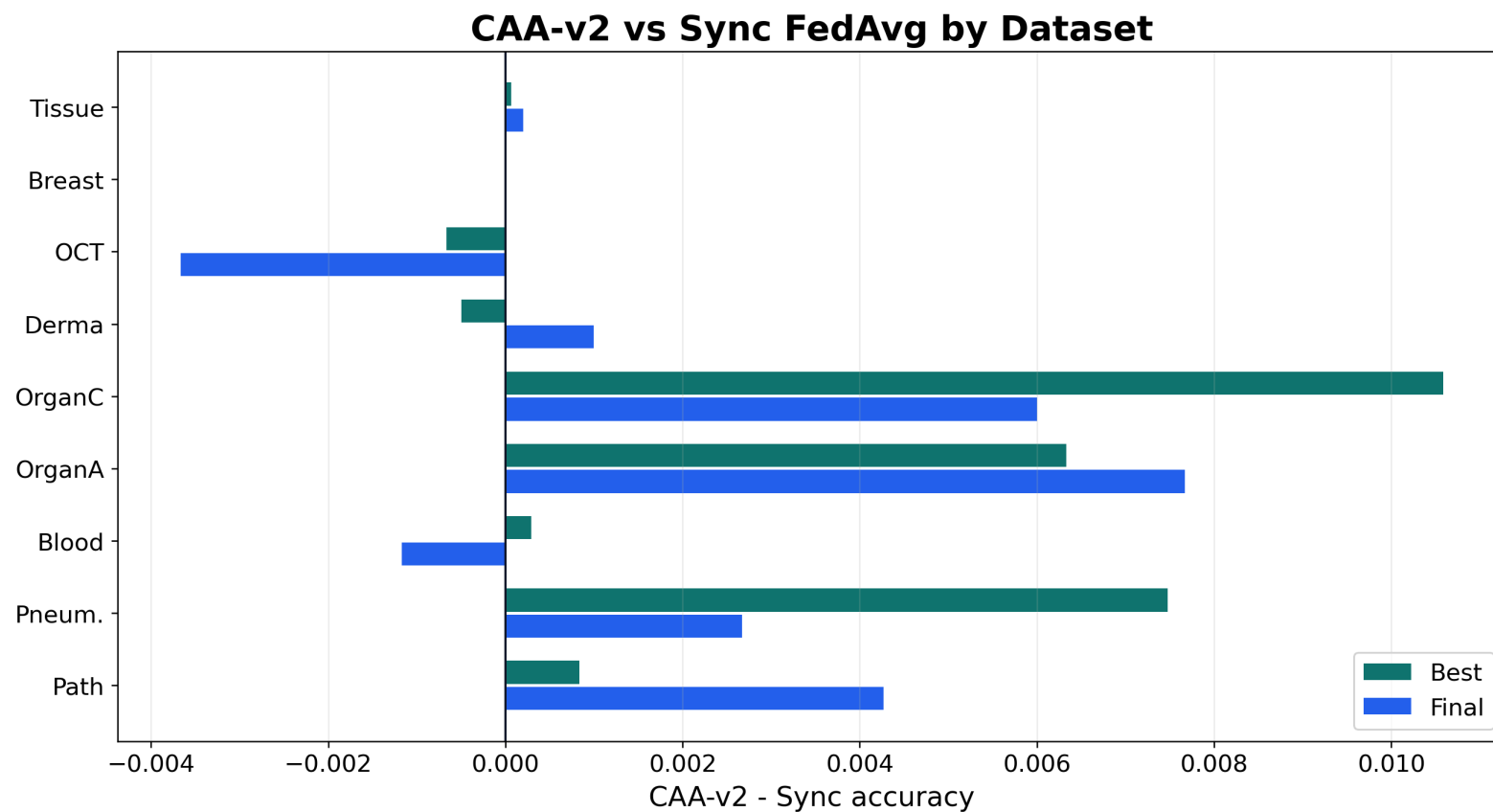


19 05 Experiment Results | Overall Dashboard

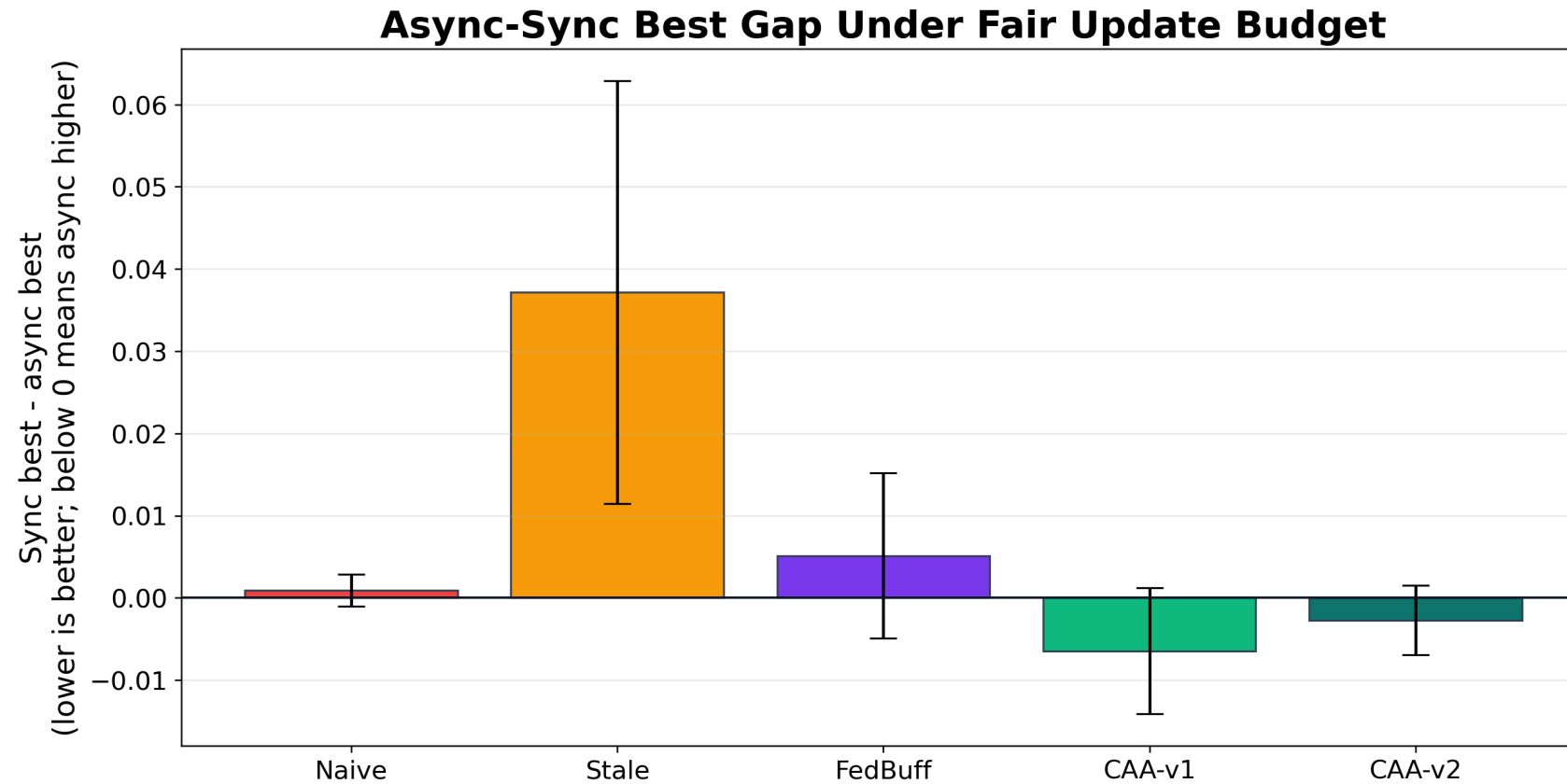


Takeaway: CAA-v2 improves mean final accuracy over Sync/Naive while keeping drop below CAA-v1 and Naive.

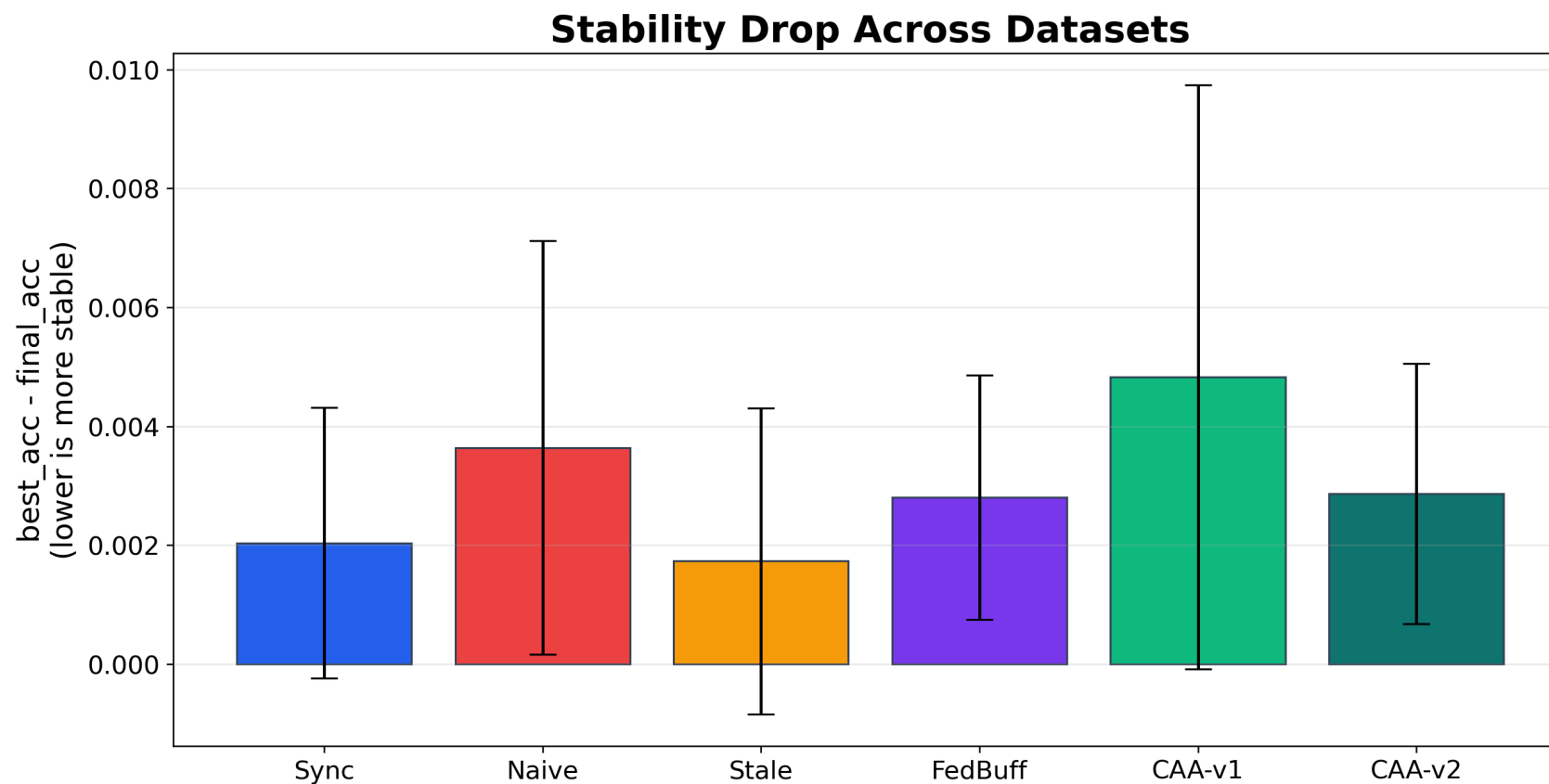
20 05 Experiment Results | CAA-v2 vs Sync



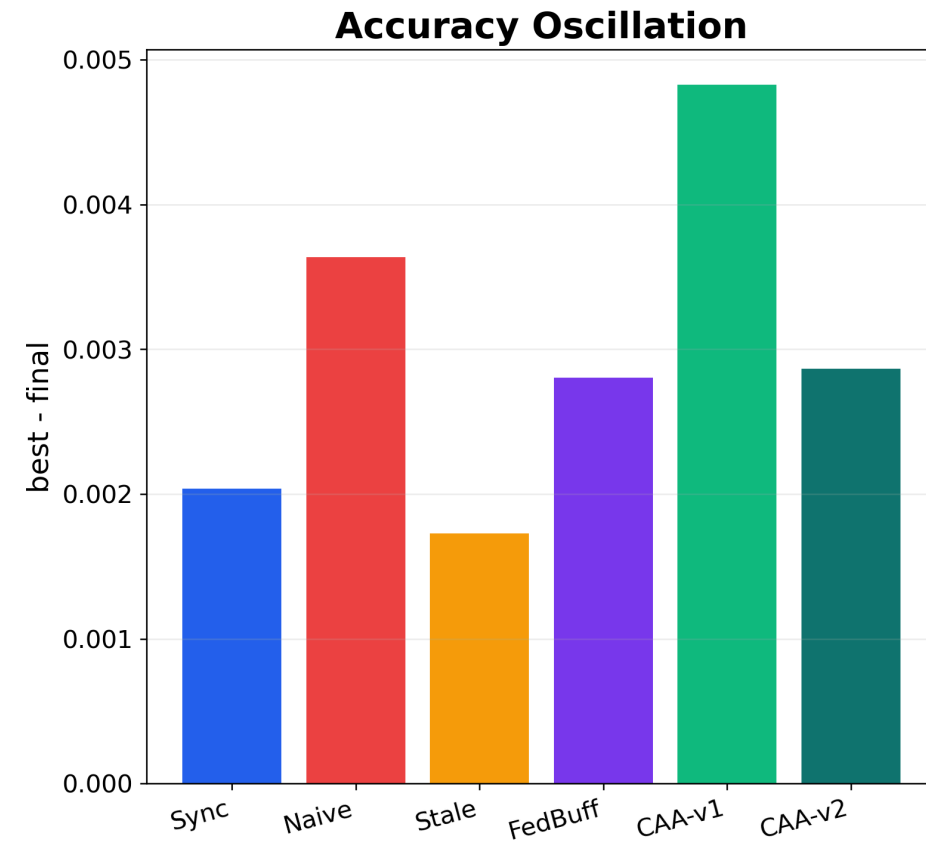
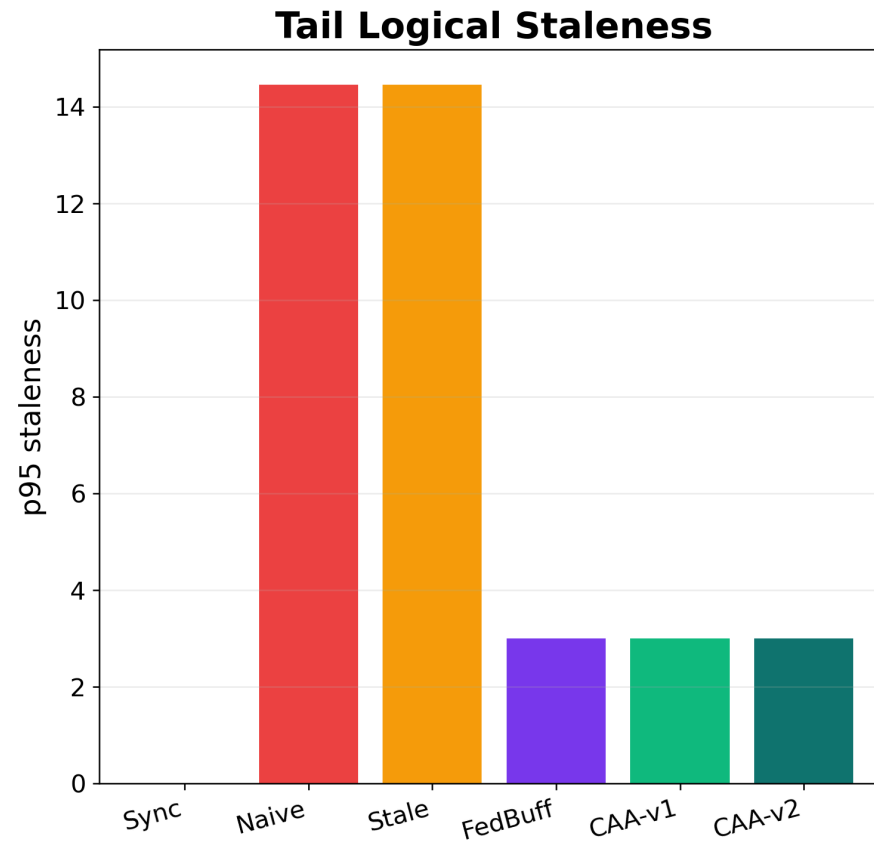
21 05 Experiment Results | Async-Sync Gap



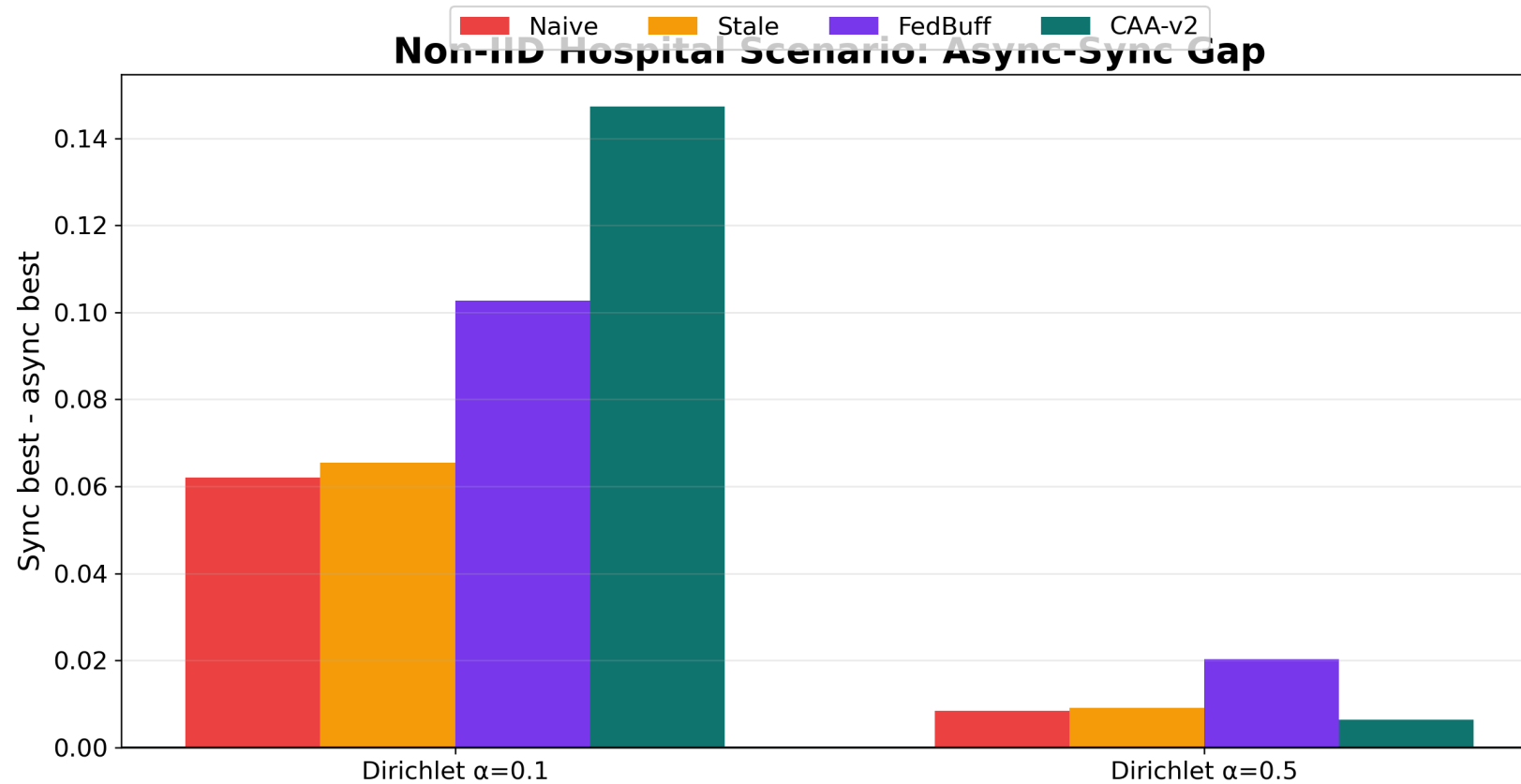
22 05 Experiment Results | Stability Drop



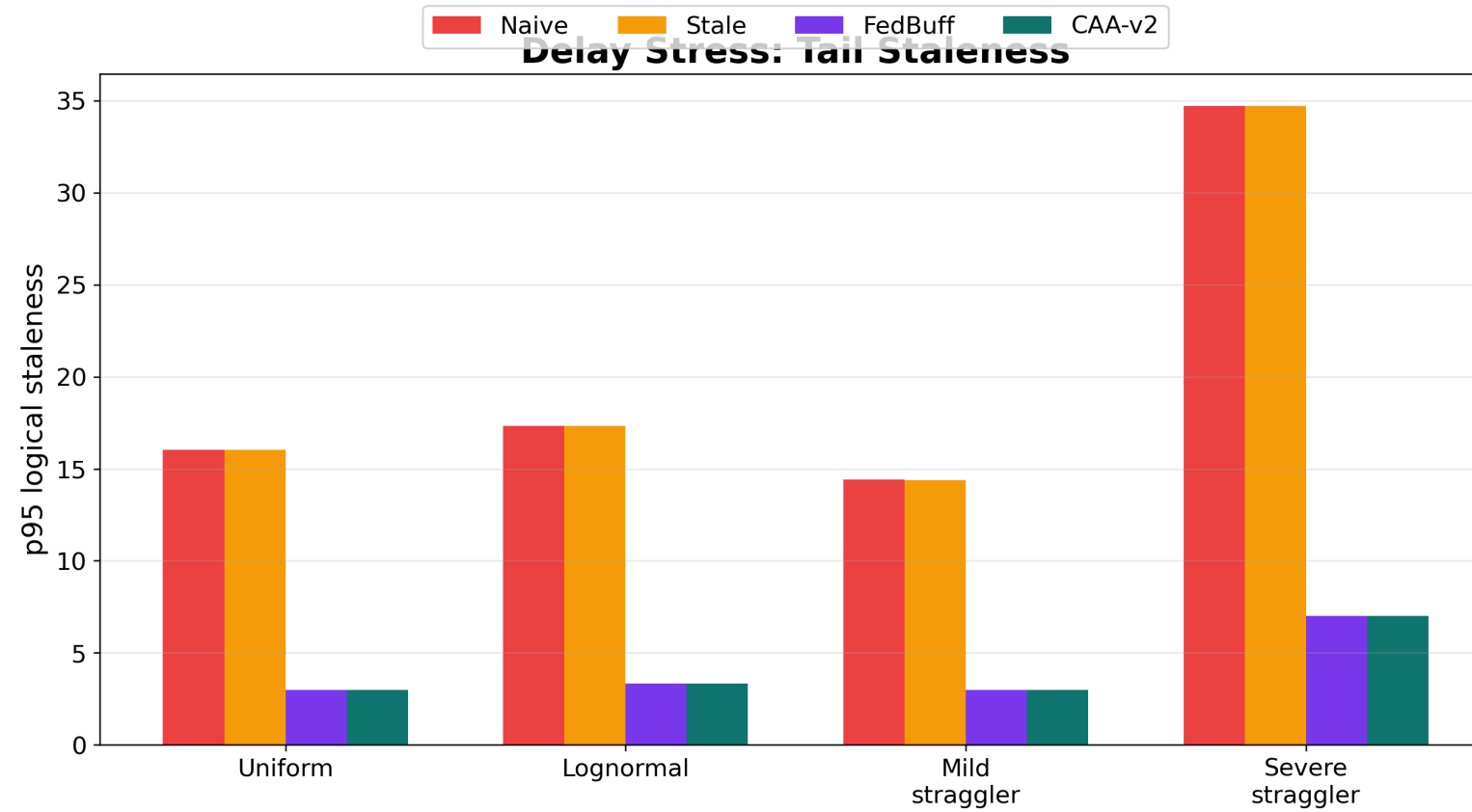
23 05 Experiment Results | System Metrics



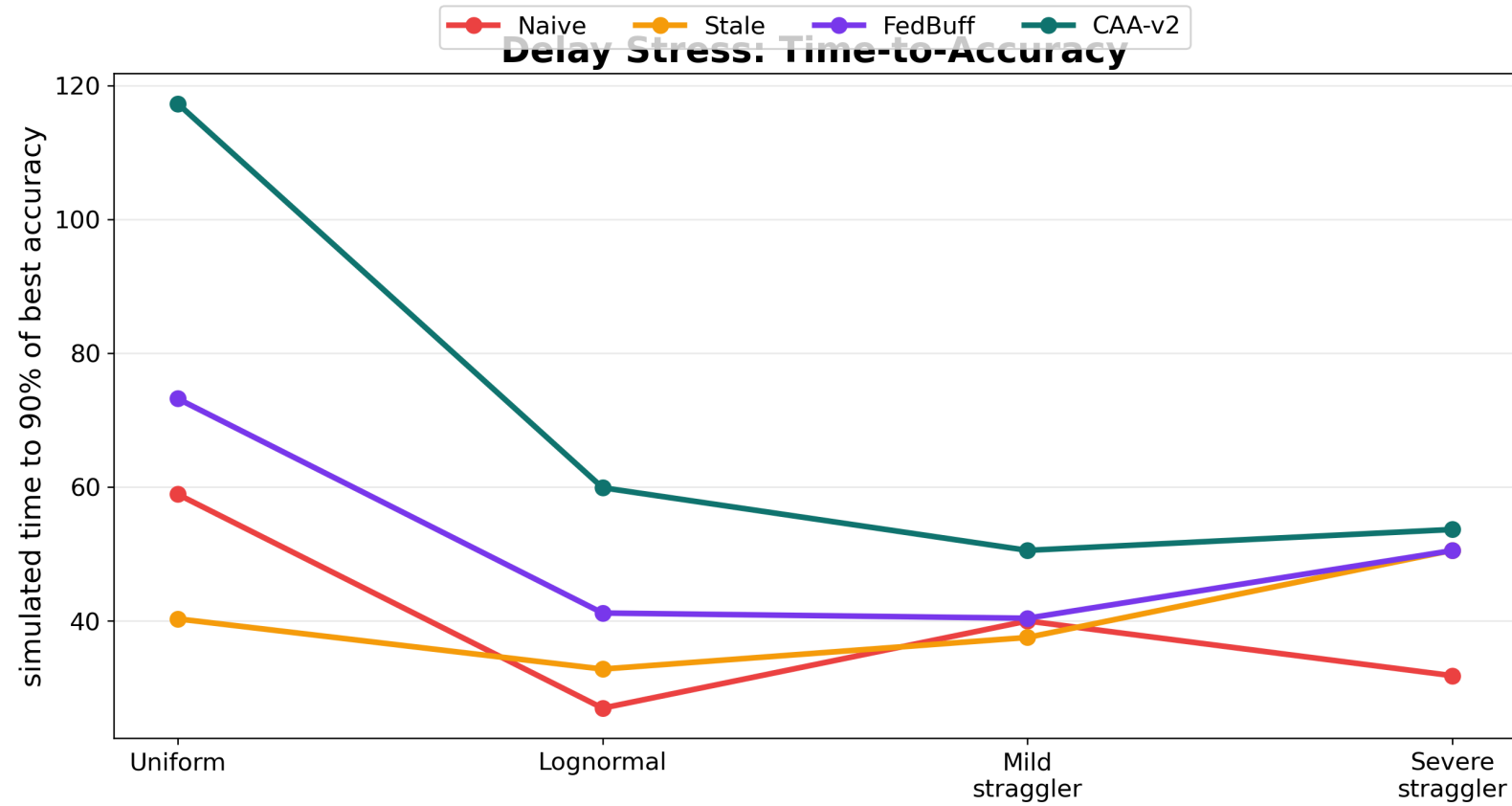
24 05 Experiment Results | Non-IID Hospital Scenario



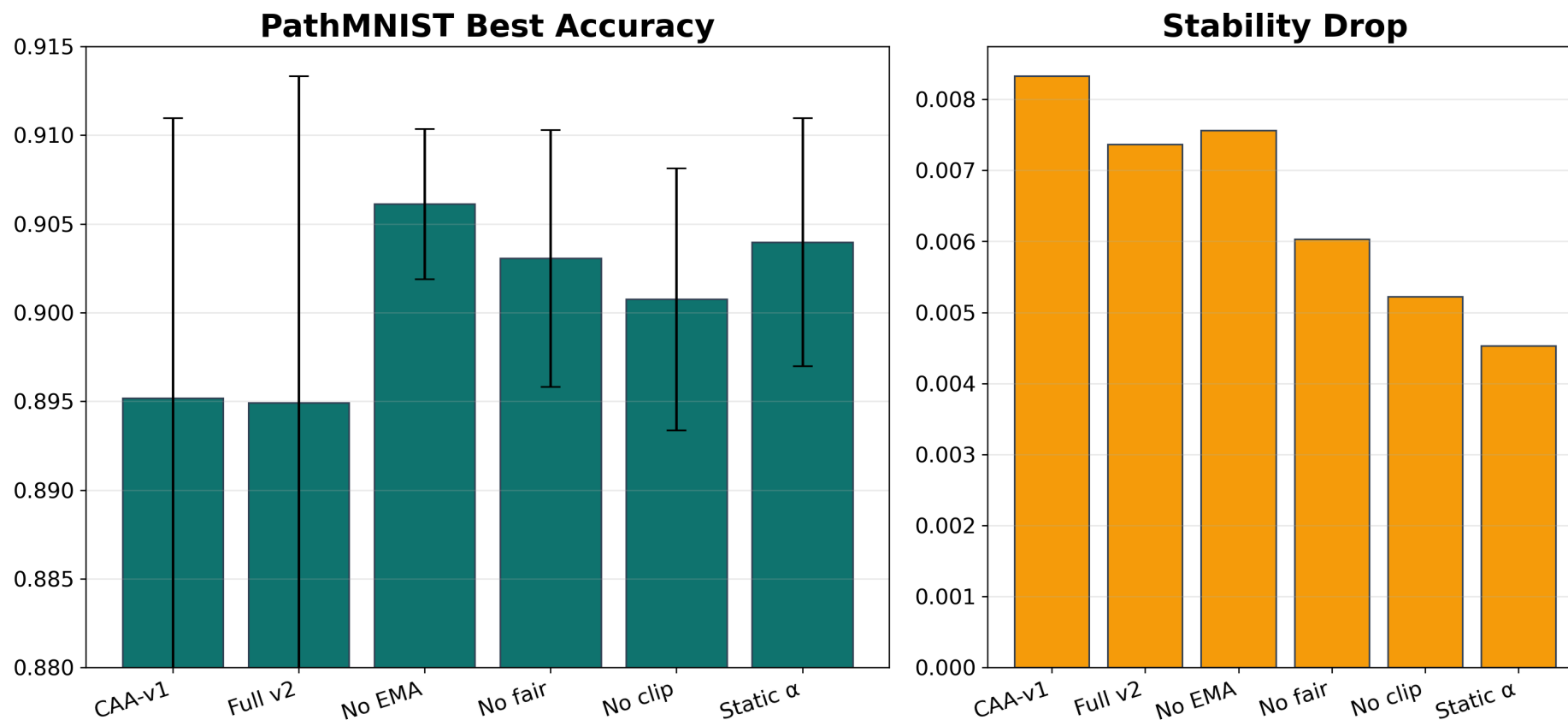
25 05 Experiment Results | Straggler Stress



26 05 Experiment Results | Simulated Time



27 05 Experiment Results | Ablation



28 05 Experiment Results | Final Interpretation

Official fair matrix:

```
CAA-v2 best  = 0.7169  
CAA-v2 final = 0.7140  
Sync final   = 0.7121  
Naive final  = 0.7096
```

Conclusion: CAA-v2 is not a universal winner, but it is more stable than Naive Async, less conservative than staleness-only, and close to or better than Sync on most datasets.

29 06 Challenges | Overview

Distributed-Systems Challenges Made Visible

**No global clock
no total physical order**

**Stale updates
old model version**

**Conflicting updates
non-IID hospital data**

**Fast-client bias
arrival dominance**

Report accuracy + staleness + simulated time + client imbalance

30 06 Challenges | No Global Clock, Stale, Conflict

Three issues happen at the same time:

```
No global clock -> cannot totally order distributed events  
Stale update    -> trained from an old global model  
Conflict update -> non-IID direction differs from others
```

CAA-v2 design: logical staleness + direction agreement + server trajectory.

31 06 Challenges | Fast Clients and Privacy Tension

Fast-client domination:

higher arrival rate from fast hospitals -> possible long-term model dominance

Privacy tension:

CAA-v2 needs delta direction / norm statistics
secure-aggregation compatibility still requires future design

Therefore, our current claim is a system simulator / design extension, not a complete privacy solution.

32 07 Conclusion | Contributions

1. Build a no-global-clock async FL simulator using logical versions.
2. Add a fair update-budget protocol.
3. Propose CAA-v2: agreement + server trajectory + fairness credit.
4. Evaluate with 9 datasets x 6 methods x 3 seeds.
5. Add distributed-systems analysis through staleness, simulated time, and client Gini.

33 07 Conclusion | Final Claim

Under a fair update budget, CAA-v2 makes clockless asynchronous FL approach Sync FedAvg across diverse MedMNIST datasets, while reducing the instability of Naive Async and avoiding the over-conservatism of staleness-only aggregation.

Aligned Chinese wording:

CAA-v2 讓無全域時鐘的非同步 FL 接近 Sync FedAvg，同時比 Naive Async 穩、比 staleness-only 不保守。

34 07 Future Work

- Compare against FedStaleWeight, SEAFL, FedPSA, and FedCompass.
- Add more multi-seed non-IID / straggler stress tests.
- Add real network traces or hospital-like traces.
- Add balanced accuracy, macro-F1, minority recall, and AUROC.
- Design secure-aggregation-compatible agreement statistics.
- Add a simplified convergence / stability analysis.

35 References

- Lamport, *Time, Clocks, and the Ordering of Events in a Distributed System*, CACM 1978.
- McMahan et al., *Communication-Efficient Learning of Deep Networks from Decentralized Data*, AISTATS 2017.
- Xie et al., *Asynchronous Federated Optimization*, 2019.
- Nguyen et al., *Federated Learning with Buffered Asynchronous Aggregation*, 2021.
- Ma et al., *FedStaleWeight*, 2024.
- Yang et al., *MedMNIST v2*, Scientific Data 2023.